DASE 7506 · SEMESTER 1 · SESSION 5

Advanced Machine Learning Methods

What is reasoning in an LLM? A pretrained base model, a generation loop, and a verifier that separates correct answers from convincing ones.

Jiayu Chen · 30 September 2026 · Raschka, *Build a Reasoning Model (From Scratch)*, chapters 1–3 · CoT and ReAct papers

SESSION 5

Today’s Four Blocks

01

Understanding Reasoning Models

Chapter 1. What reasoning means for an LLM, and the three levers that improve it.

50 min

02

Generating Text with a Pretrained LLM

Chapter 2. Qwen3 0.6B, the tokenizer, greedy generation, KV caching and compilation.

50 min

03

Evaluating Reasoning Models

Chapter 3. A math verifier — extract, normalize, check with SymPy — on MATH-500.

50 min

04

Chain-of-Thought and ReAct

The original papers, with two live-coding sessions on Qwen3 0.6B.

50 min

PART II · OVERVIEW

Where Today Sits in the Course

Part I built the LLM. Part II asks what it takes to make that LLM reason.

SESSION 5 · TODAY	SESSION 6	SESSION 7	SESSION 8
What is reasoning in an LLM?	**Inference-time scaling I: prompting and sampling**	**Inference-time scaling II: search and verifiers**	**Training reasoning models: RL and distillation**
Chapters 1–3	Chapter 4	Chapter 5	Chapters 6–8

Part I (Sessions 1–4): LLMs from scratch · Part III (Sessions 9–10): LLM agents

BLOCK I · CHAPTER 1

Understanding Reasoning Models

What “reasoning” means for an LLM, what it does not mean, and the three levers that improve it.

CHAPTER 1.1

Reasoning, in the Engineering Sense

WORKING DEFINITION

In the context of LLMs, **reasoning** means generating intermediate steps before producing the final answer.

Reasoning model

An LLM improved, through training or prompting, to produce such steps. This often raises accuracy on coding, logic puzzles and multistep math.

Chain of thought

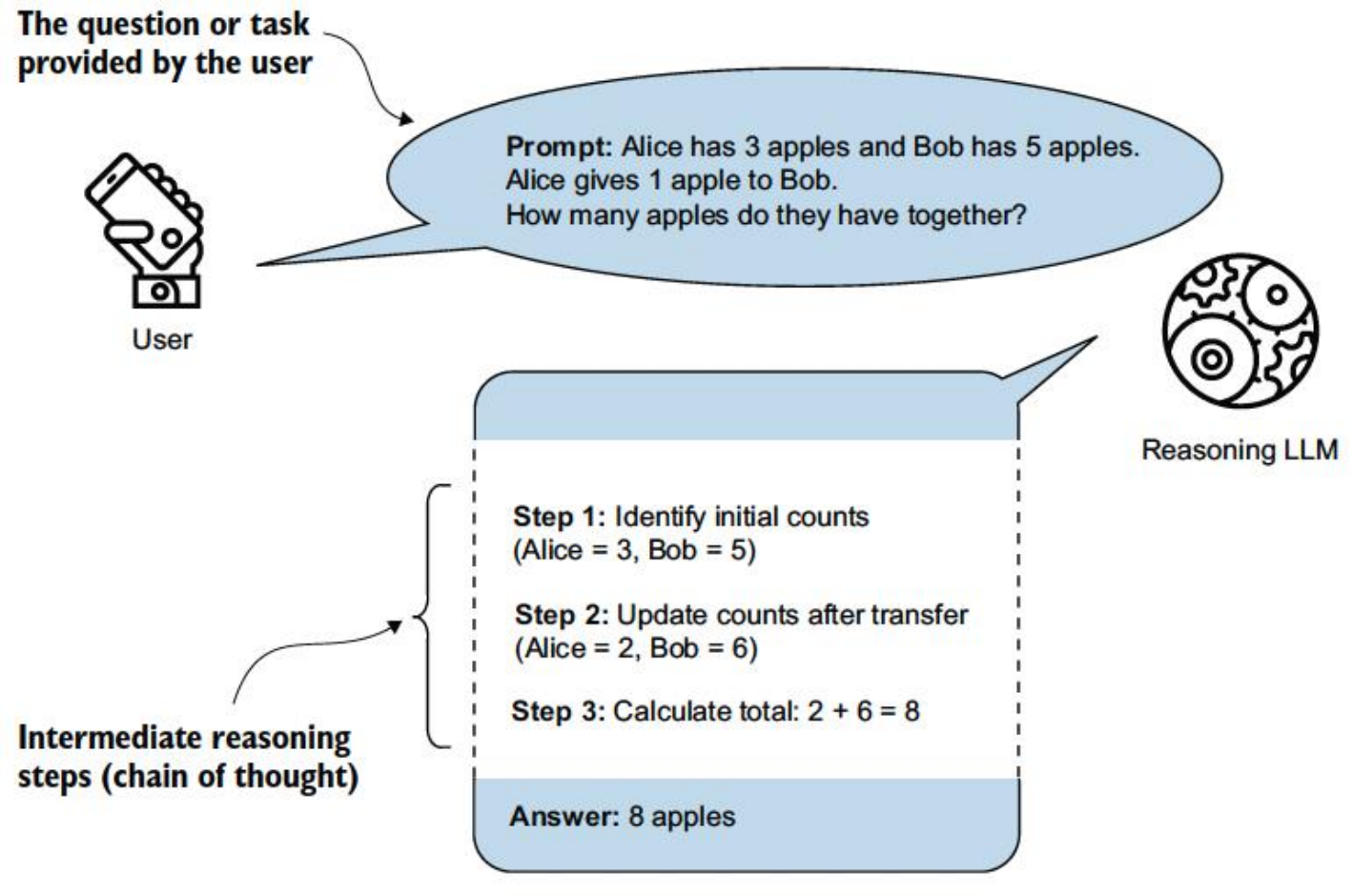
The step-by-step trace itself (CoT). It may be shown to the user, or wrapped in <think>…</think> tags and hidden.

What it does not claim

“Thinking” is an engineering label. Whether these steps mirror human reasoning is an open question the book sets aside.

CHAPTER 1.1

Reasoning, in the Engineering Sense



CHAPTER 1.1 · FIGURES 1.1–1.2

Same Question, Two Kinds of Answer

Prompt: Alice has 3 apples and Bob has 5 apples. Alice gives 1 apple to Bob. How many apples do they have together?

CONVENTIONAL LLM

```
Answer: 8 apples
```

Possibly correct, but the user cannot see how the model got there.

REASONING LLM

```
Step 1: Identify initial counts (Alice = 3, Bob = 5)
Step 2: Update counts after transfer (Alice = 2, Bob = 6)
Step 3: Calculate total: 2 + 6 = 8
Answer: 8 apples
```

The intermediate steps — the chain of thought — come before the answer.

CHAPTER 1.1

Convincing Is Not the Same as Sound

Three ways of getting from facts to a conclusion.

Symbolic logic engine

Strict, rule-based steps, like following a recipe exactly. Same input, same steps, same result. Consistency and correctness are guaranteed.

Human reasoning

Can be deterministic: we deliberately apply rules of logic or reason over an internal model of the world. We also generalize from a few examples.

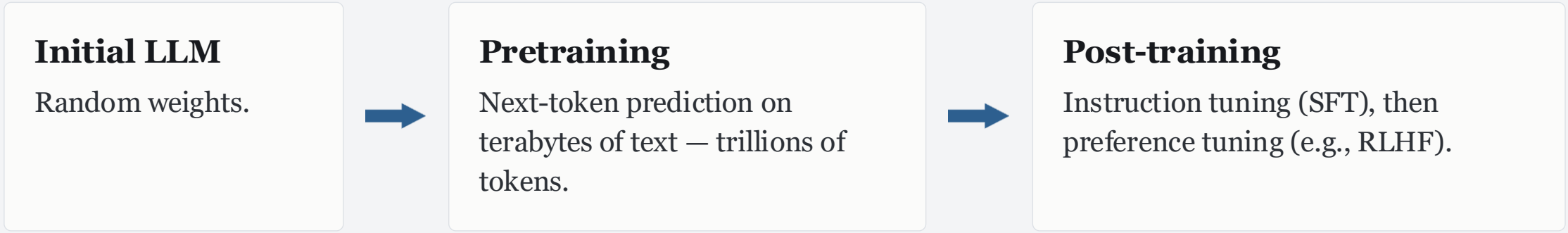
LLM reasoning

Autoregressive: one token at a time, from statistical patterns in training data. Probabilistic, with no guarantee of logical consistency — even when the steps read convincingly.

CHAPTER 1.2 · FIGURE 1.3

The Conventional Training Pipeline

What Sessions 1–4 built, and the starting point for every reasoning method.



Cost. Thousands of GPUs for months and millions of dollars. Pretrained models also show emergent abilities such as translation and code generation.

Terminology. Some papers add a mid-training stage (continued training on code, math or long context); the book folds it into pretraining.

CHAPTER 1.2 · FIGURE 1.4

What Each Stage Adds

Prompt: “Summarize the relationship between sleep and health.”

PRETRAINED

“Sleep is an essential function that allows your body and mind to recharge…”

Factual and coherent, but not a summary. Reads like an excerpt from an article.

INSTRUCTION-TUNED

“Sleep plays a critical role in maintaining physical and mental health…”

Concise, relevant, and actually a summary.

PREFERENCE-TUNED

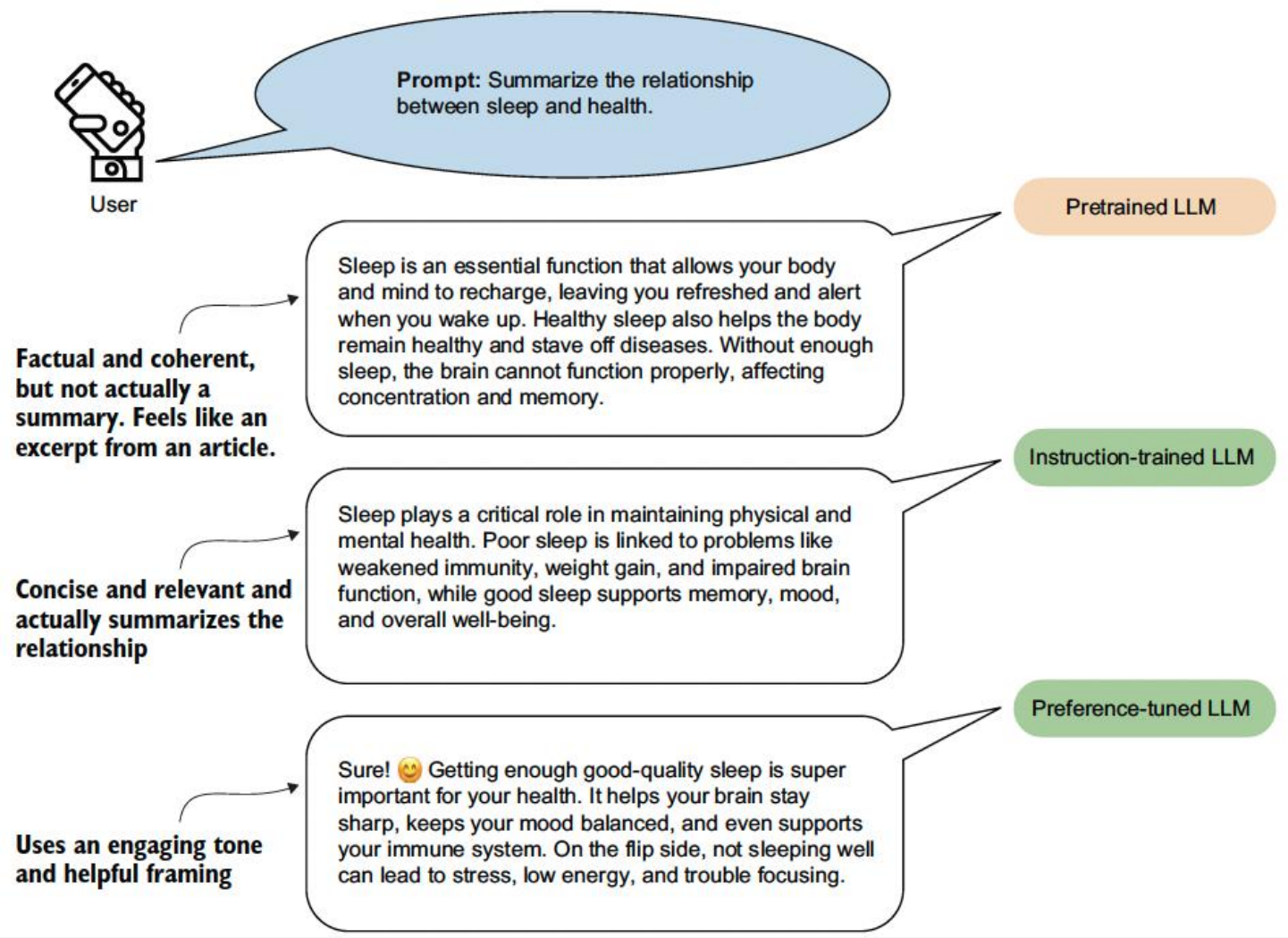
“Sure! Getting enough good-quality sleep is super important for your health…”

Same content, with an engaging tone and helpful framing.

Even an instruction-tuned model is not yet a chatbot: a chat layer adds a system prompt, conversation history and orchestration.

CHAPTER 1.2 · FIGURE 1.4

What Each Stage Adds



CHAPTERS 1.3 AND 1.6

How Reasoning Models Arrived

12 SEP 2024

OpenAI o1

“A new series of AI models designed to spend more time thinking before they respond.”

JAN 2025

DeepSeek-R1

Open weights competitive with o1, plus a technical report describing how to train such a model (arXiv 2501.12948).

12 FEB 2025

OpenAI roadmap

GPT-4.5 to be the *“last non-chain-of-thought model”*; the aim: systems that *“know when to think for a long time or not.”*

Since then, most major providers ship dedicated reasoning models, among them OpenAI’s o-series and GPT-5, Anthropic’s Claude, Google’s Gemini, xAI’s Grok, DeepSeek-R1 and Alibaba’s Qwen3.

CHAPTER 1.3 · FIGURE 1.5

Three Levers for Better Reasoning

Usually applied after pretraining and post-training; they can also be applied to the base model.

Lever	Idea	Weights	Book · Session
1 Inference-time compute scaling	Trade extra compute at inference for accuracy: chain-of-thought prompting, sampling procedures	Unchanged	Ch 4–5 · S6–7
2 Reinforcement learning	Reward outputs that earn a high reward, e.g., correct math answers or passing code	Updated	Ch 6–7 · S8
3 Distillation	Supervised fine-tuning on data generated by a larger, stronger model	Updated	Ch 8 · S8

Three Levers for Better Reasoning

Reinforcement learning for reasoning and preference tuning

In the context of developing reasoning models, it is important to distinguish the reinforcement learning (RL) approach from reinforcement learning with human feedback (RLHF), which is used during preference tuning when developing a conventional LLM. Both settings use the same underlying process (RL), but they differ primarily in how the reward is obtained and validated (human judgments for RLHF versus automated verifiers or environments for reasoning RL).

RLHF incorporates explicit human evaluations or the rankings of model outputs as reward signals, directly guiding the model toward human-preferred behaviors. In contrast, RL in the context of reasoning models typically relies on automated or environment-based reward signals, which can be more objective but less aligned with human preferences. For instance, RL in a reasoning-model development pipeline might train a model to excel at mathematical proofs by providing explicit rewards for correctness. By contrast, RLHF would involve human evaluators ranking various responses to encourage outputs that align closely with human standards and subjective preferences.

CHAPTER 1.3

Two Pairs That Are Easy to Confuse

RL for reasoning vs. RLHF

The same process — reinforcement learning. The difference is where the reward comes from.

> **RLHF:** human rankings or judgments of outputs. Steers toward human preferences; part of preference tuning.

> **Reasoning RL:** automated verifiers or environments, e.g., is the answer correct? More objective, less tied to preference.

LLM distillation vs. classic distillation

Both move knowledge from a teacher to a student. The difference is what the student sees.

> **In LLMs:** supervised fine-tuning on instruction data the teacher generated — its text only.

> **Classic deep learning:** the student learns from the teacher's outputs and its logits.

CHAPTER 1.6 · FIGURE 1.8

When to Think, and What It Costs

WORTH IT

Puzzles, advanced math, challenging coding. Agents: task decomposition, planning, choosing tools.

NOT NEEDED

Summarization, translation, knowledge-based question answering. Here reasoning is slower, wordier, and can err by "overthinking".

Why reasoning costs more

1 Longer outputs. Every generated token needs a full forward pass, so an answer twice as long needs roughly twice as many passes.

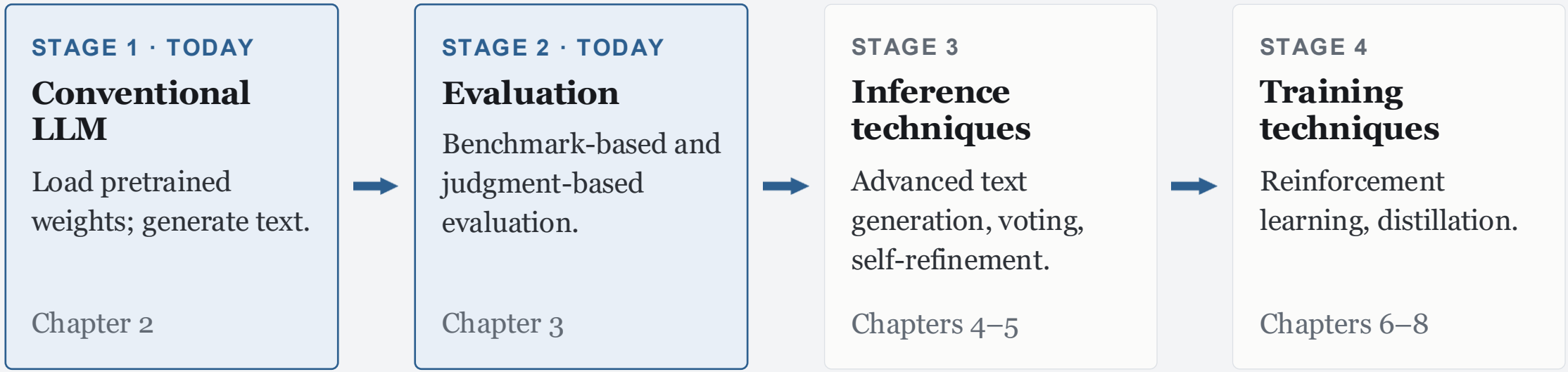
2 More calls per task. Sampling several candidates, calling tools or running a verifier multiplies the tokens processed.

This shift is also reflected in a February 12, 2025, statement from OpenAI's CEO:

We will next ship GPT-4.5, the model we called Orion internally, as our last non-chain-of-thought model. After that, a top goal for us is to unify o-series models and GPT-series models by creating systems that can use all our tools, know when to think for a long time or not, and generally be useful for a very wide range of tasks.

CHAPTER 1.7 · FIGURES 1.9–1.10

The Road Map for Part II



The Road Map for Part II

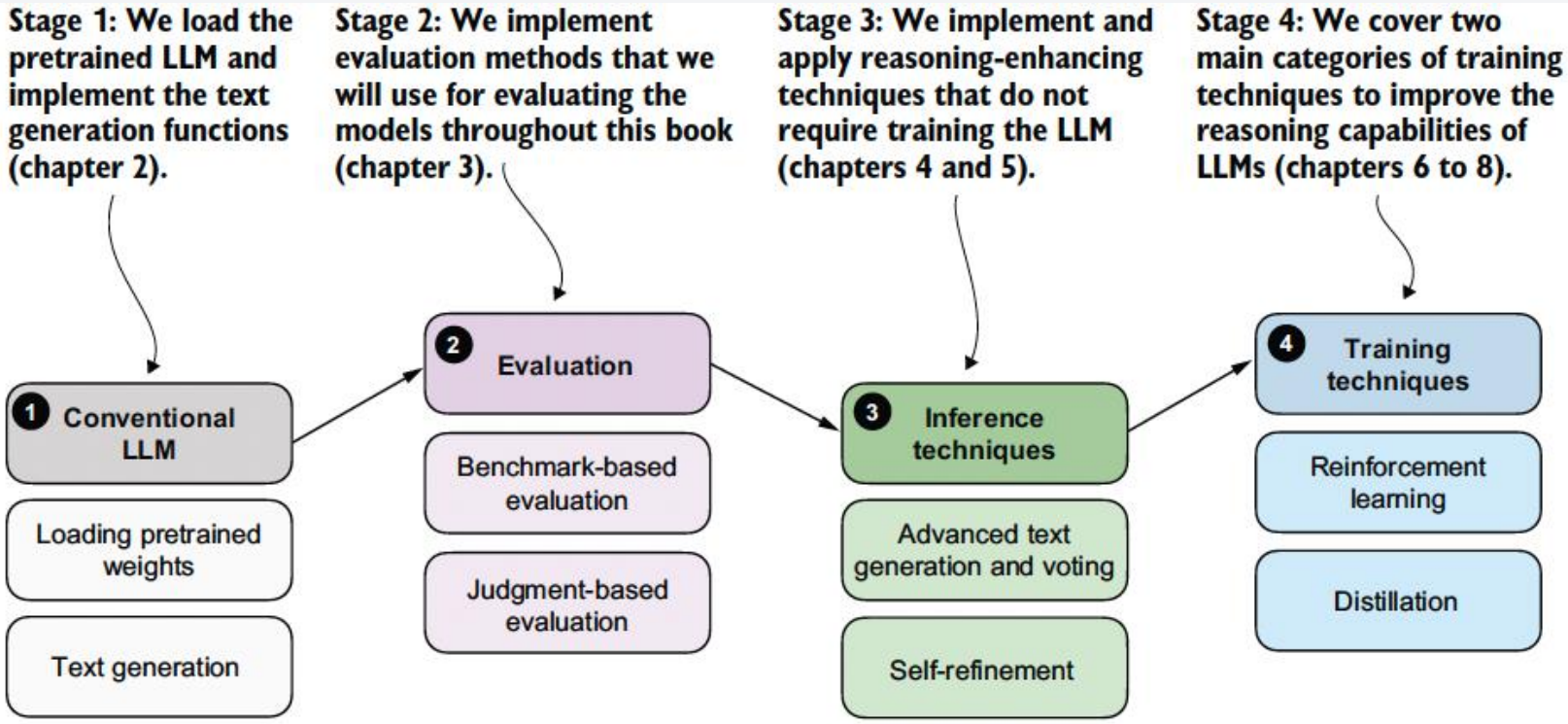


Figure 1.10 A detailed road map of the chapter-level substeps. After loading the base model, we'll cover benchmark-based and judgment-based evaluation, then inference-time methods such as advanced text generation, voting, and self-refinement, and finally training-time methods based on reinforcement learning and distillation.

Block I in Four Lines

1. Reasoning means intermediate steps — a chain of thought — before the final answer.
2. LLM reasoning is probabilistic pattern completion, not rule execution: convincing steps need not be sound.
3. Three levers: inference-time scaling (weights unchanged), reinforcement learning and distillation (weights updated).
4. Reasoning costs tokens and calls; spend it on multistep tasks, not everywhere.

BLOCK II · CHAPTER 2

Generating Text with a Pretrained LLM

Stage 1 of the road map: a base model, a tokenizer, and a loop that turns scores into text.

CHAPTERS 2.1 AND 2.5

Our Base Model: Qwen3 0.6B

Pretrained only — no instruction or preference tuning — so we can see what the reasoning methods themselves add.

Small but capable

About 0.6 billion parameters with open weights. Runs on consumer hardware, even a laptop CPU.

Has a reference twin

Qwen3 ships both a base model and an official reasoning variant — the yardstick for our evaluations in Block III.

Built from scratch

A readable pure-PyTorch reimplementation, compatible with the official weights. Runs locally: no server, no API calls.

CHAPTER 2.3

Why Not Train Our Own Base Model?

2,048	2.788M	14.8T	≈ $5.6M
NVIDIA H800 GPUs	GPU-hours for the full training run	pretraining tokens	at US$2 per GPU-hour

DeepSeek-V3, the base checkpoint behind DeepSeek-R1 — one of the few recent models with a fully transparent compute disclosure (technical report, arXiv 2412.19437).

THE BOOK'S ANALOGY

To learn how engines work, start with a Volkswagen Beetle, not a Ferrari. A small model follows the same architecture as state-of-the-art LLMs, and the reasoning methods are the ones larger models use.

CHAPTERS 2.2–2.3

Setting Up for the Lab

OPTION A · PIP

```
pip install -r requirements.txt
# or just this chapter's packages:
pip install "torch>=2.10.0" \
    "tokenizers>=0.22.2" \
    reasoning-from-scratch
```

OPTION B · UV (THE AUTHOR'S CHOICE)

```
git clone --depth 1 \
  https://github.com/rasbt/reasoning-from-scratch.git
cd reasoning-from-scratch
uv run jupyter lab
# creates the environment on first run
```

> **reasoning-from-scratch** is the book's own package: all chapter code plus utilities. Python 3.10 or newer.

> **Hardware.** Chapters 2–5 run in reasonable time on a CPU; later chapters benefit from a GPU (free tiers: Google Colab, Lightning AI).

CHAPTER 2.4 · FIGURE 2.5

From Text to Token IDs and Back

Every prompt, reasoning trace and answer passes through the tokenizer.

“Explain large language models.”

⬇ `tokenizer.encode(text)`

Ex	plain	␣large	␣language	␣models	.
840	20772	3460	4128	4119	13

⬇ `LLM: input IDs → output IDs, one at a time`

␣Large	␣language	␣models	␣are	␣advanced	…
20286	4128	4119	525	10847	

⬇ `tokenizer.decode(ids)`

“Large language models are advanced…”

CHAPTER 2.4 · FIGURE 2.6

A Tokenization Round Trip

```
from reasoning_from_scratch.qwen3 import (
    download_qwen3_small, Qwen3Tokenizer)

download_qwen3_small(kind="base",
    tokenizer_only=True, out_dir="qwen3")
tokenizer_path = Path("qwen3") / "tokenizer-base.json"
tokenizer = Qwen3Tokenizer(tokenizer_file_path=tokenizer_path)

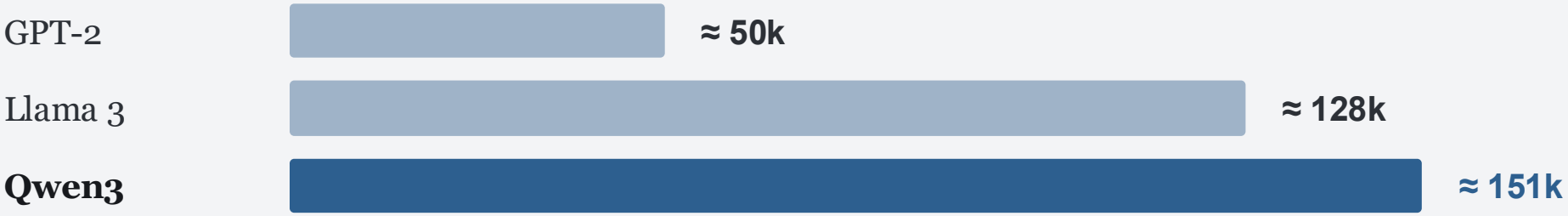
ids = tokenizer.encode("Explain large language models.")
for i in ids:
    print(f"{i} --> {tokenizer.decode([i])}")
```

```
840 --> Ex
20772 --> plain
3460 --> ␣large
4128 --> ␣language
4119 --> ␣models
13 --> .
```

“Explain” becomes “Ex” + “plain”: byte pair encoding (BPE) mixes whole words and subword pieces.

CHAPTER 2.4

Vocabulary Size Is a Trade-off



Larger vocabulary

Bigger embedding and output layers, and a somewhat higher per-token cost for next-token probabilities.

Smaller vocabulary

More words split into subwords, so longer sequences. Doubling the tokens roughly doubles the compute.

Exercise 2.1. Encode words that do not exist, and words in another language — try Chinese. How does the tokenizer cope?

CHAPTER 2.5 · LISTING 2.1

Loading the Pretrained Weights

```
device = get_device()
# CUDA, then MPS, then XPU, else CPU

download_qwen3_small(kind="base",
    tokenizer_only=False, out_dir="qwen3")
# qwen3-0.6B-base.pth: 1433 MiB

model_path = Path("qwen3") / "qwen3-0.6B-base.pth"
model = Qwen3Model(QWEN_CONFIG_06_B)
model.load_state_dict(torch.load(model_path))
model.to(device)
```

get_device() also enables TF32 tensor cores on recent NVIDIA GPUs — slightly different rounding, faster matrix multiplies.

Debug on the CPU first, then remove the override and rerun on the GPU.

Familiar pattern from Session 3: build with random weights, then overwrite them with a pretrained state dict.

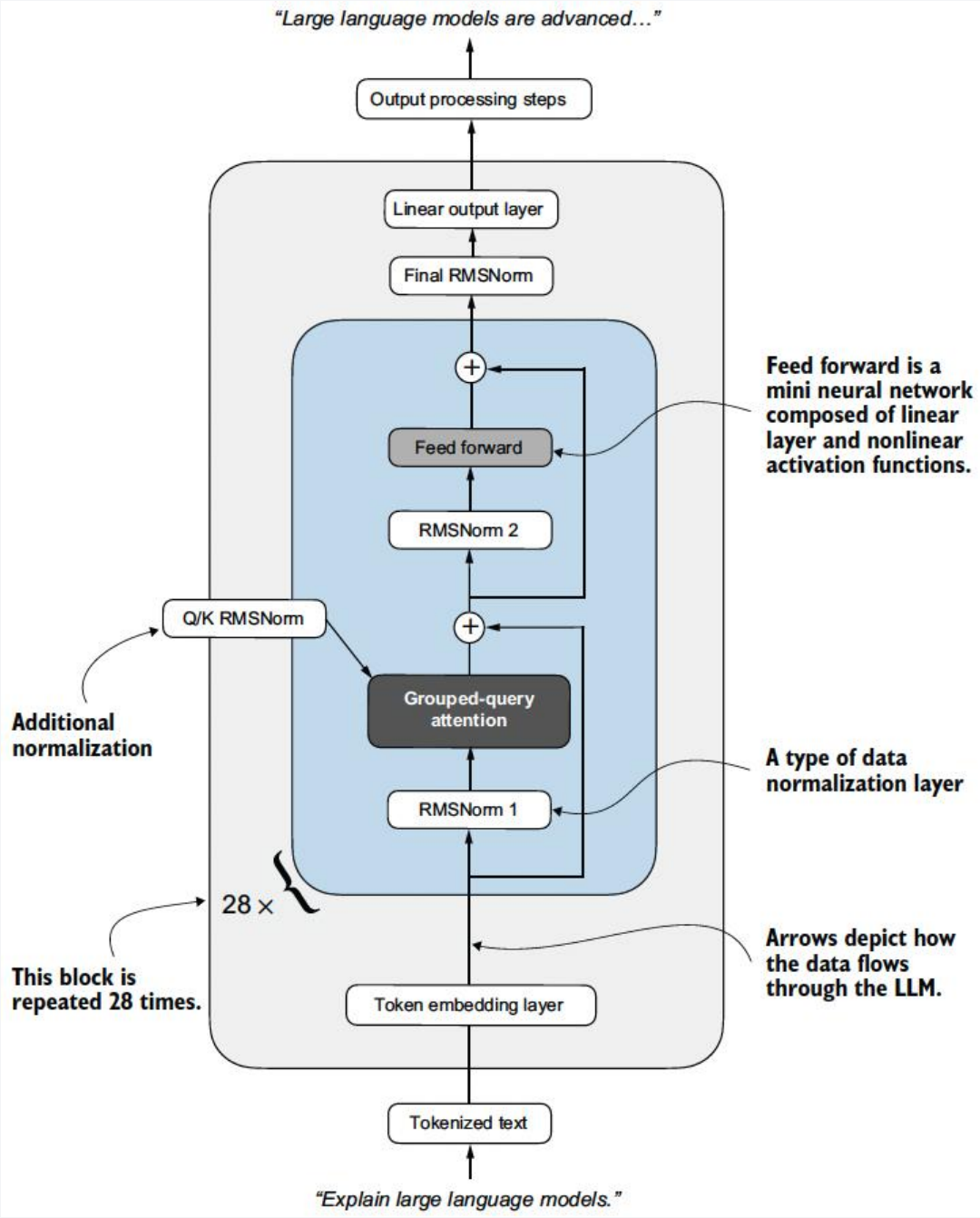
CHAPTER 2.5 · FIGURE 2.8

Qwen3 0.6B Against the GPT-2 You Built

Component	GPT-2 small (Session 3)	Qwen3 0.6B (today)
Vocabulary	50,257	151,936
Embedding size	768	1,024
Transformer blocks	12	28
Attention	Multi-head, 12 heads	Grouped-query: 16 query heads share 8 key/value heads; RMSNorm on Q and K
Normalization	LayerNorm	RMSNorm
Feed-forward	768 → 3,072 → 768, GELU	Gated (SwiGLU): fc1, fc2 1,024 → 3,072; fc3 3,072 → 1,024
Positions	Learned absolute embeddings	Rotary embeddings (RoPE) inside attention

CHAPTER 2.5 · FIGURE 2.8

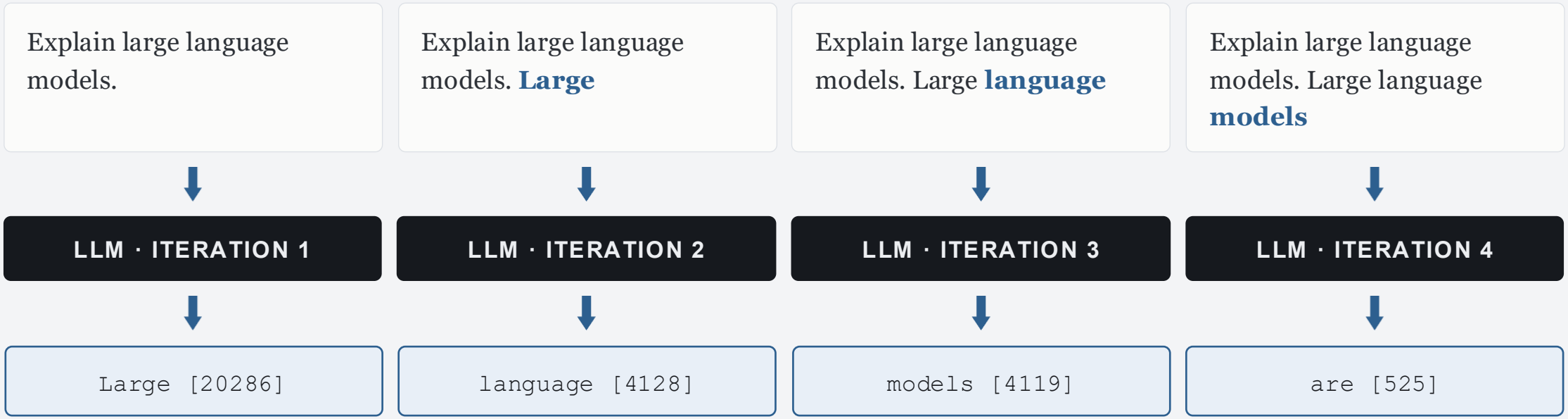
Qwen3 0.6B Against the GPT-2 You Built



CHAPTER 2.6 · FIGURES 2.10 AND 2.14

One Token at a Time

Autoregressive generation: each new token is appended to the input of the next iteration.



Inside One Iteration

```
ids = torch.tensor(tokenizer.encode(prompt))
ids = ids.unsqueeze(0).to(device)
# add a batch dimension: [1, 6]

with torch.inference_mode():
    out = model(ids).squeeze(0)
# out.shape: [6, 151936]

last = out[-1]
next_id = torch.argmax(last, dim=-1, keepdim=True)
print(tokenizer.decode(next_id.tolist()))
# next_id = tensor([20286]) → " Large"
```

1 Six tokens in, a 6 × 151,936 matrix out: one row per position, one score per vocabulary entry.

2 Row *i* scores the token that follows position *i*. Generation needs only the last row.

3 argmax picks the highest score: ID 20286, “Large”. Always taking the maximum is **greedy decoding**.

CHAPTER 2.7 · LISTING 2.2

The Minimal Generation Loop

```
@torch.inference_mode()
def generate_text_basic_stream(
        model, token_ids, max_new_tokens, eos_token_id=None):
    model.eval()
    for _ in range(max_new_tokens):
        out = model(token_ids)[:, -1]
        next_token = torch.argmax(out, dim=-1, keepdim=True)
        if (eos_token_id is not None
                and torch.all(next_token == eos_token_id)):
            break
        yield next_token
        token_ids = torch.cat([token_ids, next_token], dim=1)
```

[:, -1] scores at the last position, for every sequence in the batch.

argmax greedy: the single highest-scoring token.

yield streams each token as soon as it exists.

torch.cat appends it; the whole sequence is re-run next step.

CHAPTER 2.7

Knowing When to Stop

WITHOUT EOS_TOKEN_ID · 100 NEW TOKENS

```
Large language models are artificial intelligence systems that
can understand, generate, and process human language, […] and
even creating creative content.<|endoftext|>Human language is a
complex and dynamic system that has evolved over millions of
years…
```

WITH EOS_TOKEN_ID=TOKENIZER.EOS_TOKEN_ID

```
Large language models are artificial intelligence systems that
can understand, generate, and process human language, […] and
even creating creative content.
```

Why it keeps going

<|endoftext|> (ID 151643) separates documents in pretraining. After it, a base model simply starts a new “document”.

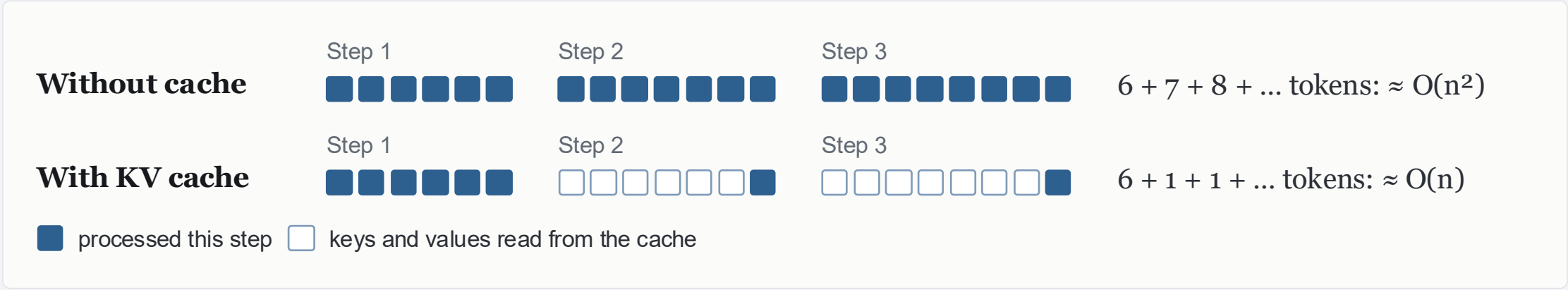
“Inference”

Running a trained, fixed model forward. Not statistical inference: nothing is estimated.

CHAPTER 2.8 · FIGURE 2.16

KV Caching: Stop Recomputing the Past

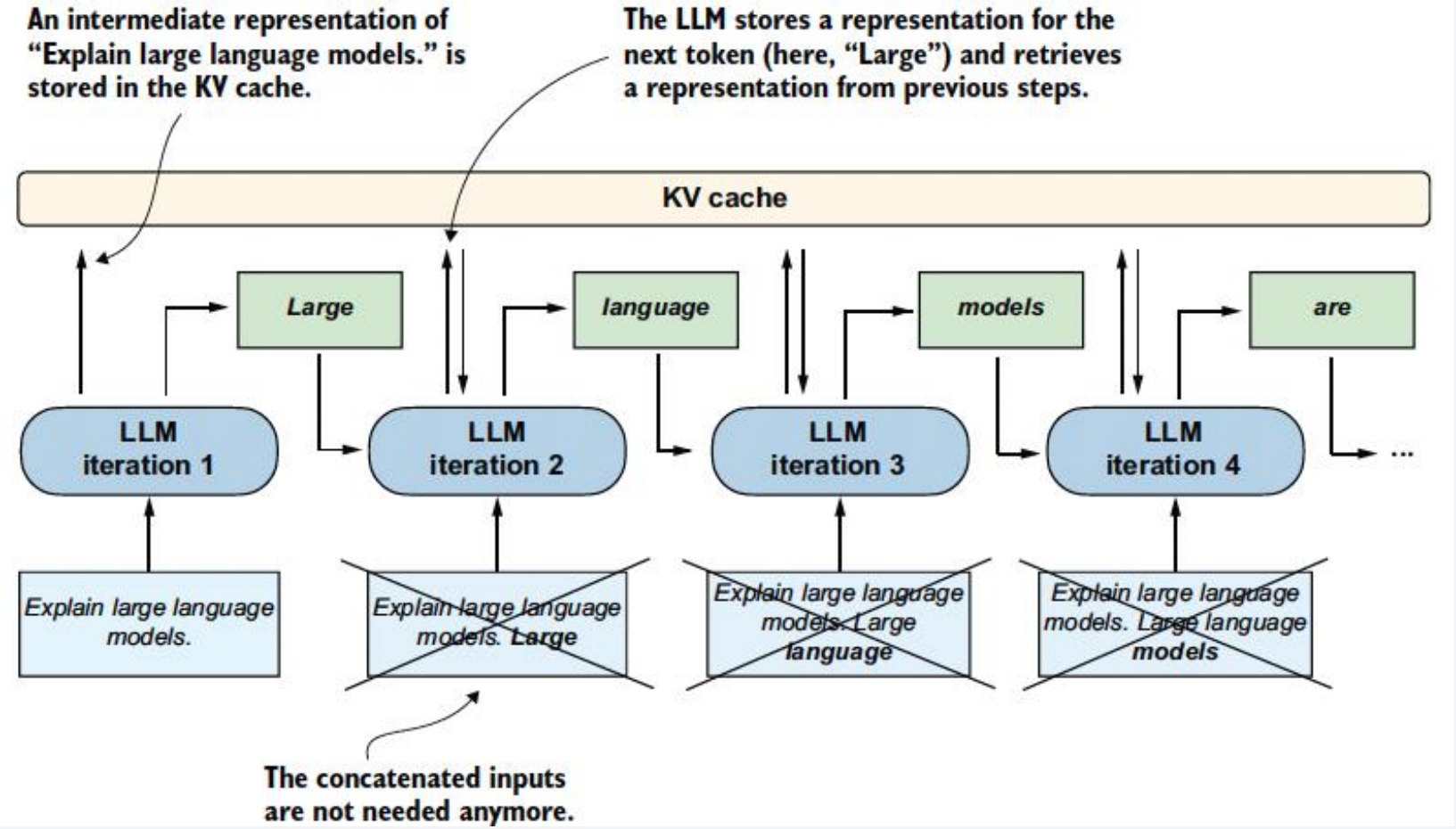
Keys and values of earlier tokens never change, so compute them once, store them, and reuse them.



The generated text is identical with and without the cache — a built-in correctness check.

CHAPTER 2.8 · FIGURE 2.16

KV Caching: Stop Recomputing the Past



CHAPTER 2.8 · LISTING 2.4

Adding the Cache to the Loop

```
cache = KVCache(n_layers=model.cfg["n_layers"])
model.reset_kv_cache()

out = model(token_ids, cache=cache)[:, -1] # first call: the whole prompt
for _ in range(max_new_tokens):
    next_token = torch.argmax(out, dim=-1, keepdim=True)
    if (eos_token_id is not None
            and torch.all(next_token == eos_token_id)):
        break
    yield next_token
    out = model(next_token, cache=cache)[:, -1] # later calls: one token
```

> **No more torch.cat** on the input: only next_token goes in; earlier keys and values come from the cache.

> **About 5–6× faster** on the book's Mac Mini M4 CPU, with the same output text.

CHAPTER 2.9 · TABLE 2.1

Compilation, and What the Numbers Say

torch.compile(model) turns groups of operations into optimized kernels. The first call compiles (a slow warm-up); time the runs after it.

Tokens / sec	M4 CPU	M4 GPU	H100
Regular	5	27	51
Regular, compiled	6	43	164
KV cache	28	41	48
KV cache, compiled	68	71	141

Qwen3 0.6B, one prompt (batch size 1). Bold = fastest per device.

Why the cache barely helps on the H100

The cache grows with torch.cat instead of preallocated K and V: memory-friendly, but slow on GPUs.

A 0.6B model at batch size 1 leaves the GPU underused.

Compilation, and What the Numbers Say

```
major, minor = map(int, torch.__version__.split(".")[:2])
if (major, minor) >= (2, 8):
    # This avoids retriggering model recompilations
    # in PyTorch 2.8 and newer
    # if the model contains code like self.pos = self.pos + 1
    torch._dynamo.config.allow_unspec_int_on_nn_module = True

model_compiled = torch.compile(model)
```

```
Warm-up run

Time: 11.68 sec
3 tokens/sec

------------------------------

 Large language models are artificial intelligence systems that can
understand, generate, and process human language, enabling them to
perform a wide range of tasks, from answering questions to writing
articles, and even creating creative content.

Timed run 1:

Time: 6.78 sec
6 tokens/sec

------------------------------

 Large language models are artificial intelligence systems that can
understand, generate, and process human language, enabling them to
perform a wide range of tasks, from answering questions to writing
articles, and even creating creative content.

Timed run 2:

Time: 6.80 sec
6 tokens/sec
```

Block II in Four Lines

1. Qwen3 0.6B base: small, open-weight, rebuilt in plain PyTorch, with an official reasoning twin for comparison.
2. The tokenizer maps text to token IDs and back with BPE; vocabulary size trades model size against sequence length.
3. Generation: one forward pass per token, keep the last row, argmax, append, stop at <|endoftext|>.
4. KV caching and torch.compile together: 5 → 68 tokens/sec on a Mac Mini M4 CPU.

BLOCK III · CHAPTER 3

Evaluating Reasoning Models

Stage 2 of the road map: telling models that solve problems from models that merely sound convincing.

CHAPTER 3.1 · FIGURE 3.1

Four Ways to Evaluate an LLM

BENCHMARK-BASED · MORE QUANTITATIVE

Multiple choice

Choose among given options; score against the key.

Verifiers · today

Extract the final answer; check it against the reference.

JUDGMENT-BASED · MORE QUALITATIVE

Leaderboards

Rank models by preference votes on their outputs.

LLM judges

Another LLM scores the response.

Why verifiers for reasoning

Free-form, multistep answers — yet the final result can be checked deterministically.

The same checker becomes the verifiable reward for RL in Session 8.

> **Limit:** only domains that are easy to verify, such as math and code.

CHAPTER 3.1 · FIGURE 3.3

A Math Verifier in Eight Steps

1. Load the pretrained LLM
2. Generate an answer
3. Extract the final boxed answer
4. Normalize it
5. Verify equivalence with the ground truth
6. Grade, including multi-part answers
7. Load the MATH-500 dataset
8. Evaluate: run steps 2–6 over the dataset

ONE ANSWER, TRACED

```
… Final Answer: \boxed{\dfrac{14}{3}}
```

⬇ 3 · extract

```
\dfrac{14}{3}
```

⬇ 4 · normalize

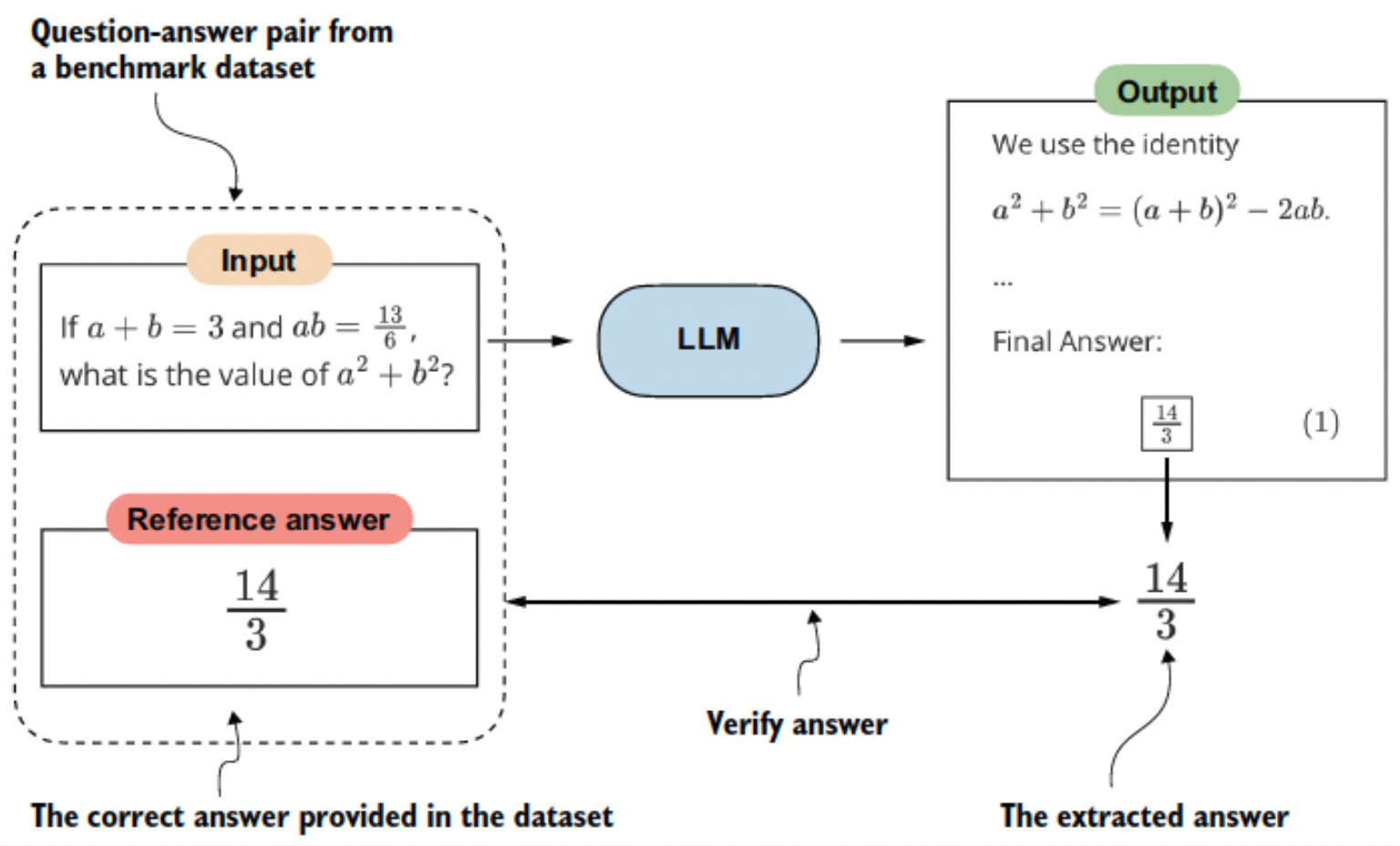
```
(14)/(3)
```

⬇ 5 · compare with ground truth 14/3

True — correct

CHAPTER 3.1 · FIGURE 3.3

A Math Verifier in Eight Steps



CHAPTERS 3.2–3.3 · LISTINGS 3.1–3.3 · STEPS 1–2

Load and Generate

```
model, tokenizer = load_model_and_tokenizer(
    which_model="base",  # or "reasoning"
    device=device, use_compile=False)

prompt = (r"If $a+b=3$ and $ab=\tfrac{13}{6}$, "
          r"what is the value of $a^2+b^2$?")
generated_text = generate_text_stream_concat(
    model, tokenizer, prompt, device,
    max_new_tokens=2048, verbose=True)
```

BASE MODEL OUTPUT (RENDERED)

… we can use the identity $a^2 + b^2 = (a + b)^2 - 2ab$.

$a^2 + b^2 = 9 - 13/3 = 14/3$

```
**Final Answer:** \boxed{\dfrac{14}{3}}
```

Correct.

> A base model writing a chain of thought: the Qwen3 technical report says chain-of-thought data was part of pretraining.

> "reasoning" loads the official Qwen3 reasoning model: our reference, not the model we build.

CHAPTER 3.4 · LISTINGS 3.4–3.5 · STEP 3

Extract the Last Box

Listing 3.4 Extracting answer boxes

```
def get_last_boxed(text):
    boxed_start_idx = text.rfind(r"\boxed")
    if boxed_start_idx == -1:
        return None

    current_idx = boxed_start_idx + len(r"\boxed")

    while current_idx < len(text) and text[current_idx].isspace():
        current_idx += 1

    if current_idx >= len(text) or text[current_idx] != "{":
        return None

    current_idx += 1
    brace_depth = 1
    content_start_idx = current_idx

    while current_idx < len(text) and brace_depth > 0:
        char = text[current_idx]
        if char == "{":
            brace_depth += 1
        elif char == "}":
            brace_depth -= 1
        current_idx += 1

    if brace_depth != 0:
        return None

    return text[content_start_idx:current_idx-1]
```

Find the last occurrence of "\boxed".

Get the position after "\boxed".

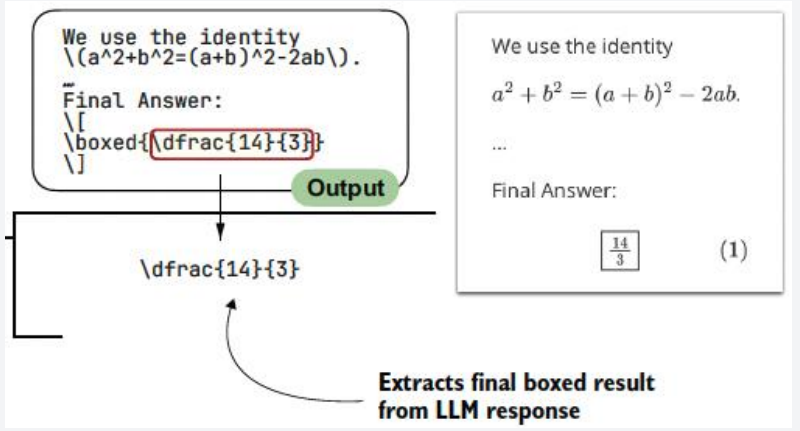
Skip any whitespace after "\boxed".

Expect an opening brace "{".

Parse the braces with nesting.

Account for unbalanced braces.

Extract content inside the outermost braces.



Listing 3.5 Extracting the final answer candidate

```
import re

RE_NUMBER = re.compile(
    r"-?(?:\d+/\d+|\d+(?:\.\d+)?(?:[eE][+-]?\d+)?)"
)

def extract_final_candidate(text, fallback="number_then_full"):

    result = ""

    if text:
        boxed = get_last_boxed(text.strip())

        if boxed:
            result = boxed.strip().strip("$ ")

        elif fallback in ("number_then_full", "number_only"):
            m = RE_NUMBER.findall(text)
            if m:
                result = m[-1]
            elif fallback == "number_then_full":

                result = text
    return result
```

Regular expression for extracting numeric values from the text

Default return value if nothing matches

Prefer the last boxed expression if present.

If there is no boxed expression, try the fallback.

Use the last number.

Else return the full text if no number is found.

CHAPTER 3.5 · LISTING 3.6 · STEP 4

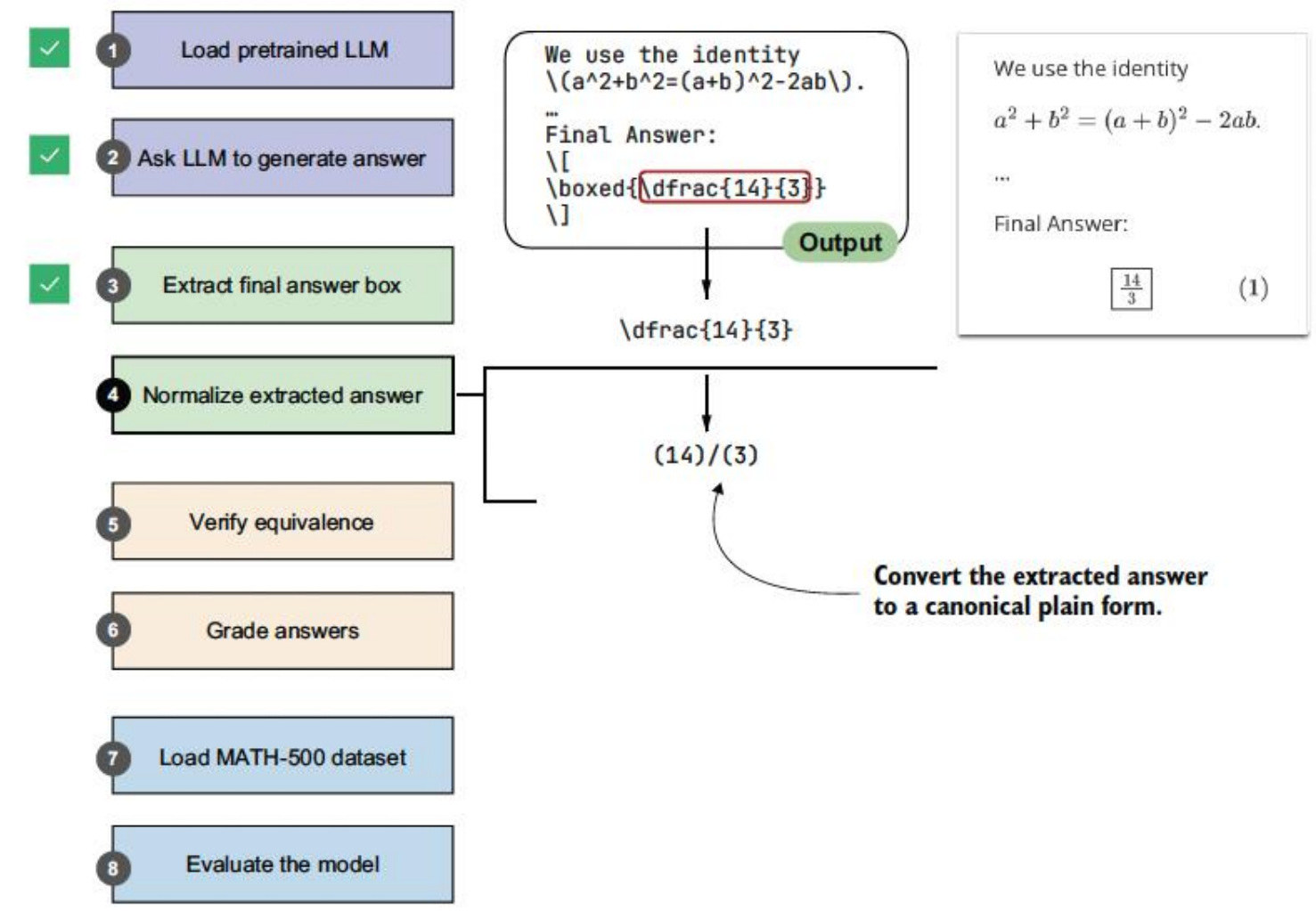
Normalize to One Canonical Form

One value, many spellings: normalize_text() rewrites each into a plain, calculator-style string.

What it handles	Before	After
Chat special tokens	<|assistant|>14/3	14/3
Multiple-choice labels	c. 3	3
Degree markers	90^\circ	90
\text{...} and math delimiters	\text{\[\frac{14}{3}\]}	(14)/(3)
LaTeX variants	\left(\dfrac{3}{4}\right)	((3)/(4))
Roots and fractions	\frac{\sqrt{8}}{2}	(sqrt(8))/(2)
Exponents, superscripts	2^{3} · 2²	2**3 · 2**2
Separators, %, $	1,234 · 50\% · $14/3$	1234 · 50 · 14/3

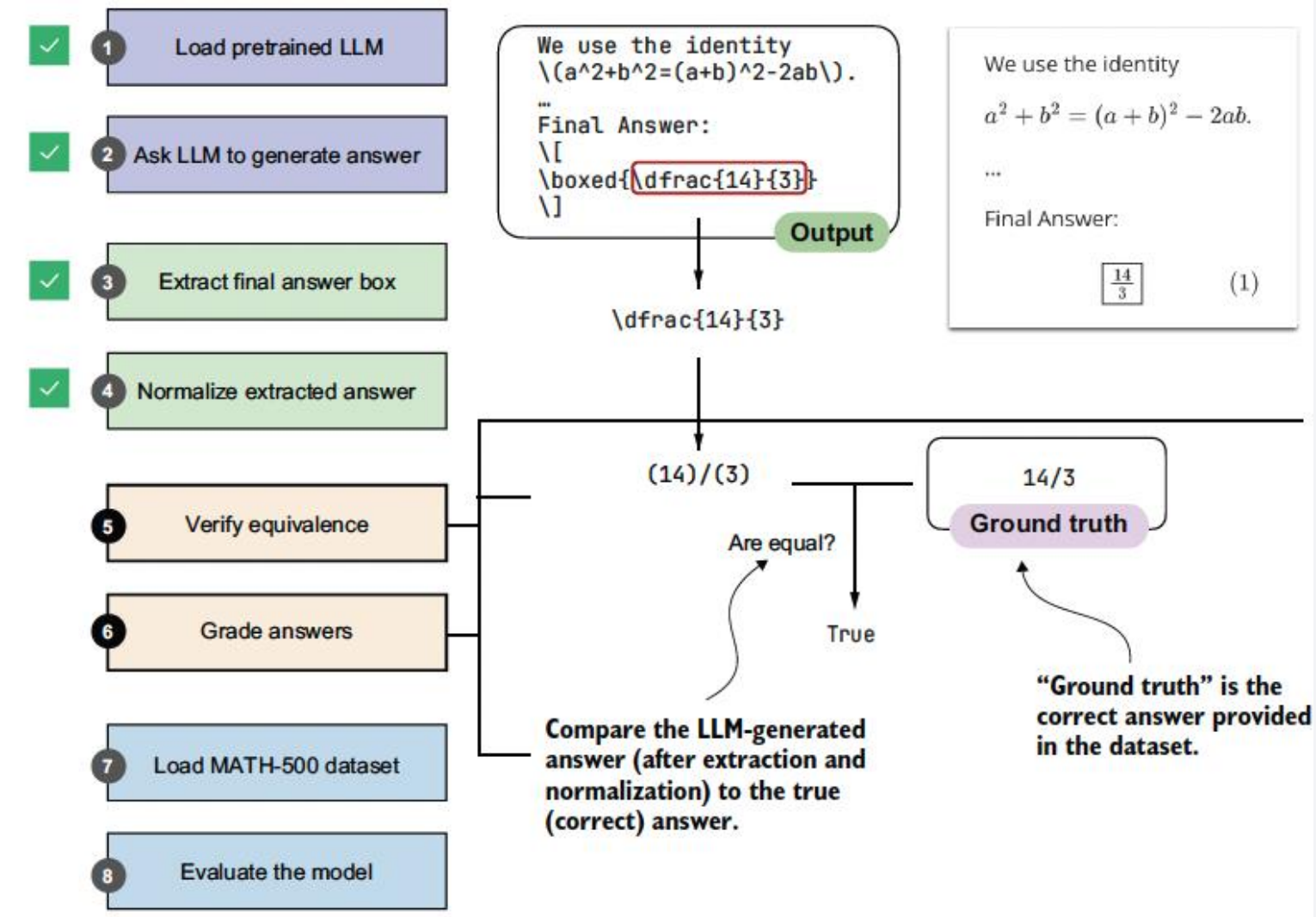
CHAPTER 3.5 · LISTING 3.6 · STEP 4

Normalize to One Canonical Form



CHAPTER 3.6 · LISTINGS 3.7–3.8 · STEP 5

Equivalence, Not String Equality



Equivalence, Not String Equality

Listing 3.7 Using the SymPy parser for a mathematical equality check

```
from sympy.parsing import sympy_parser as spp
from sympy.core.sympify import SympifyError
from sympy.polys.polyerrors import PolynomialError
from tokenize import TokenError

def sympy_parser(expr):
    if expr is None or len(expr) > 2000:
        return None

    try:
        return spp.parse_expr(
            expr,
            transformations=(
                *spp.standard_transformations,

                spp.implicit_multiplication_application,
            ),
            evaluate=True,
        )
    except (SympifyError, SyntaxError, TypeError, AttributeError,
            IndexError, TokenError, ValueError, PolynomialError):
        return None
```

To avoid crashing on long garbage responses

Standard transformations like handling parentheses

Allow omitted multiplication symbols (e.g., 2y → 2*y).

Evaluate during parsing so simple constants simplify (e.g., 2+3 → 5).

Equivalence, Not String Equality

```
def equality_check(expr_gtruth, expr_pred):
    if expr_gtruth == expr_pred:
        return True
    gtruth, pred = (sympy_parser(expr_gtruth),
                    sympy_parser(expr_pred))
    if gtruth is not None and pred is not None:
        try:
            return simplify(gtruth - pred) == 0
        except (SympifyError, TypeError):
            pass
    return False
```

A	B	Result
13/4.	(13)/(4)	True
0.5	(1)/(2)	True
14/3	15/3	False
(14/3, 2/3)	(14/3, 4/6)	False

> The tuple should be **True**: equality_check handles single expressions only. Step 6 fixes this.

sympy_parser() calls SymPy's parse_expr with implicit multiplication (2y → 2*y) and evaluate=True (2+3 → 5); it returns None for unparsable or over-long input.

CHAPTER 3.7 · LISTINGS 3.9–3.11 · STEP 6

Grading Multi-Part Answers

```
def grade_answer(pred_text, gt_text):
    if pred_text is None or gt_text is None:
        return False
    gt_parts = split_into_parts(normalize_text(gt_text))
    pred_parts = split_into_parts(normalize_text(pred_text))
    if not gt_parts or len(gt_parts) != len(pred_parts):
        return False
    return all(equality_check(gt, pred)
               for gt, pred in zip(gt_parts, pred_parts))
```

split_into_parts() turns “(a, b)” or “[a, b]” into [a, b]; anything else stays one part. Order matters.

Prediction	Truth	Expected
(2, 1)	(1, 2)	False
(1, 2, 3)	(1, 2)	False
0.3333333333	1/3	False
1,234/2	617	True
50%	1/2	False
90^\circ	90	True
2^2	2**2	True

From Listing 3.11 — the grader passes all 16 cases.

CHAPTER 3.7 · LISTINGS 3.9–3.11 · STEP 6

Grading Multi-Part Answers

Listing 3.9 A helper function to split tuple-like expressions

```
def split_into_parts(text):
    result = [text]

    if text:
        if (
            len(text) >= 2
            and text[0] in "([" and text[-1] in ")]"
            and "," in text[1:-1]
        ):
            items = [p.strip() for p in text[1:-1].split(",")]
            if all(items):
                result = items
    else:
        result = []

    return result
```

Check if text looks like a tuple or list, e.g., “(a, b)” or “[a, b]”.

Split on commas inside brackets and strip whitespace.

If text is empty, return an empty list.

EXERCISE 3.1 · SOME ANSWERS

Where the Grader Gets It Wrong

Prediction	Ground truth	Graded	Why
[0, 1)	(0, 1]	True	Brackets only trigger splitting; open and closed endpoints are ignored
x = 3	3	False	The “x =” prefix is not stripped, so it never equals 3
5\text{ cm}	5	False	Units are not removed
(3, pi/2)	\left(3, \frac{\pi}{2} \right)	False	\pi is never mapped to pi, so only an exact string match succeeds

A verifier is a specification. Once it is the RL reward (Session 8), every false positive is a loophole the model can learn to exploit.

The MATH-500 Benchmark

- 500 problems sampled from the MATH dataset; the split comes from OpenAI's PRM800K and is on Hugging Face as HuggingFaceH4/MATH-500.
- Big enough to be meaningful, small enough to re-run after every change.
- Held out: later chapters train on a non-overlapping subset of MATH.
- Standard in the reasoning literature, and harder than GSM8K.

```
{'answer': '\\left( 3, \\frac{\\pi}{2} \\right)',
 'level': 2,
 'problem': 'Convert the point $(0,3)$ in rectangular coordinates to polar '
            'coordinates.  Enter your answer in the form $(r,\\theta),$ where '
            '$r > 0$ and $0 \\le \\theta < 2 \\pi.$',
 'solution': 'We have that $r = \\sqrt{0^2 + 3^2} = 3.$  Also, if we draw the '
             'line connecting the origin and $(0,3),$ this line makes an
    angle '
             'of $\\frac{\\pi}{2}$ with the positive $x$-axis.\n'
             '\n'
             '[asy]\n'
             'unitsize(0.8 cm);\n'
             '\n'
             'draw((-0.5,0)--(3.5,0));\n'
             'draw((0,-0.5)--(0,3.5));\n'
             'draw(arc((0,0),3,0,90),red,Arrow(6));\n'
             '\n'
             'dot((0,3), red);\n'
             'label("$(0,3)$", (0,3), W);\n'
             'dot((3,0), red);\n'
             '[/asy]\n'
             '\n'
             'Therefore, the polar coordinates are $\\boxed{\\left( 3, '
             '\\frac{\\pi}{2} \\right)}.$',
 'subject': 'Precalculus',
 'unique_id': 'test/precalculus/807.json'}
```

CHAPTER 3.9 · LISTING 3.13

The Prompt Template Matters

Listing 3.13 Function to render a prompt template for math evaluation

```python
def render_prompt(prompt):
    template = (
        "You are a helpful math assistant.\n"
        "Answer the question and write the final result on a new line as:\n"
        "\\boxed{ANSWER}\n\n"
        f"Question:\n{prompt}\n\nAnswer:"
    )
    return template
```

```
You are a helpful math assistant.
Answer the question and write the final result on a new line as:
\boxed{ANSWER}

Question:
If $a+b=3$ and $ab=\tfrac{13}{6}$, what is the value of $a^2+b^2$?

Answer:
```

ONE EXAMPLE, TWO PROMPTS

No template: a long step-by-step answer ending in 14/3 — correct. With the template: \boxed{10} — short, fast, wrong.

REPORTED IN THE BOOK

No template: base model ~50% better, reasoning model ~40% worse. “Problem:” instead of “Question:”: base ~20% better, reasoning ~30% worse.

CHAPTER 3.9 · LISTINGS 3.14–3.15 · STEP 8

The Evaluation Loop

Listing 3.14 Demo function to run the evaluation pipeline

```
def mini_eval_demo(model, tokenizer, device):
    ex = {
        "problem": "Compute 1/2 + 1/6.",
        "answer": "2/3"
    }
    prompt = render_prompt(ex["problem"])
    gen_text = generate_text_stream_concat(
        model, tokenizer, prompt, device,
        max_new_tokens=64,
    )
    pred_answer = extract_final_candidate(gen_text)
    is_correct = grade_answer(
        pred_answer, ex["answer"]
    )

    print(f"Device: {device}")
    print(f"Prediction: {pred_answer}")
    print(f"Ground truth: {ex['answer']}")
    print(f"Correct: {is_correct}")
```

Test the example with "problem" and "answer" fields.

1. Apply prompt template

2. Generate response

3. Extract and normalize answer

4. Grade answer

The Evaluation Loop

```
num_correct = 0
with open(out_path, "w", encoding="utf-8") as f:
    for i, row in enumerate(math_data, start=1):
        prompt = render_prompt(row["problem"])
        gen_text = generate_text_stream_concat(model, tokenizer,
            prompt, device, max_new_tokens=max_new_tokens)
        extracted = extract_final_candidate(gen_text)
        is_correct = grade_answer(extracted, row["answer"])
        num_correct += int(is_correct)
        # the book also logs problem, answer, full text
        record = {"index": i, "extracted": extracted,
                  "correct": bool(is_correct)}
        f.write(json.dumps(record) + "\n")
acc = num_correct / len(math_data)
```

Smoke-test first. “Compute 1/2 + 1/6.” The base model answers 1/3: extracted correctly, simply wrong.

Log everything. One JSONL line per problem, so you can read the failures, not just count them.

Leave room. max_new_tokens=2048, so the reasoning variant is not cut off.

CHAPTER 3.9 · TABLE 3.1

Accuracy Costs Tokens

QWEN3 0.6B BASE

15.3%

MATH-500 accuracy, all 500 problems

13.3 min

to evaluate on an NVIDIA H100

QWEN3 0.6B REASONING VARIANT

50.8%

MATH-500 accuracy, all 500 problems

185.4 min

to evaluate on an NVIDIA H100

About 3.3× the accuracy for about 14× the time. Batching (batch size 128) cuts the runs to 3.3 and 14.6 min.

Block III in Four Lines

1. Verifiers suit reasoning models: free-form steps, but a final answer that can be checked deterministically.
2. The pipeline: generate, extract the last \boxed{}, normalize, check equivalence with SymPy, grade each part.
3. A grader is a specification; its errors become the model's incentives once it serves as an RL reward.
4. On MATH-500, the reasoning variant scores 50.8% against the base model's 15.3%, at about 14× the evaluation time.

BLOCK IV · ORIGINAL PAPERS + LIVE CODING · 50 MIN

Chain-of-Thought and ReAct

From “Let’s think step by step” to Thought → Action → Observation: three papers, two live-coding sessions on Qwen3 0.6B.

BLOCK IV · READING LIST

Three Papers, One Thread

WEI ET AL. · NEURIPS 2022

Chain-of-Thought Prompting Elicits Reasoning in Large Language Models

Few-shot exemplars that show their working.

KOJIMA ET AL. · NEURIPS 2022

Large Language Models are Zero-Shot Reasoners

No exemplars: one trigger sentence.

YAO ET AL. · ICLR 2023

ReAct: Synergizing Reasoning and Acting in Language Models

Interleave reasoning with actions on a tool.

CoT papers · 12 min | **Live coding 1 · 12 min** | **ReAct paper · 12 min** | **Live coding 2 · 12 min**

WEI ET AL. 2022 · FIGURE 1

Chain-of-Thought Prompting

STANDARD PROMPTING

```
Q: Roger has 5 tennis balls. He buys 2 more cans of
tennis balls. Each can has 3 tennis balls. How many
tennis balls does he have now?
A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to
make lunch and bought 6 more, how many apples do they
have?
```

```
A: The answer is 27. wrong
```

CHAIN-OF-THOUGHT PROMPTING

```
Q: Roger has 5 tennis balls. … How many tennis balls
does he have now?
A: Roger started with 5 balls. 2 cans of 3 tennis
balls each is 6 tennis balls. 5 + 6 = 11. The answer
is 11.

Q: The cafeteria had 23 apples. …
```

```
A: The cafeteria had 23 apples originally. They used
20 to make lunch. So they had 23 - 20 = 3. They bought
6 more apples, so they have 3 + 6 = 9. The answer is
9. correct
```

What the Paper Proposes

Prompt with triples ⟨input, chain of thought, output⟩, where a chain of thought is *"a series of intermediate natural language reasoning steps that lead to the final output."*

Decomposition

More steps, so more computation, for harder problems.

Interpretability

A window for debugging where the reasoning went wrong.

Generality

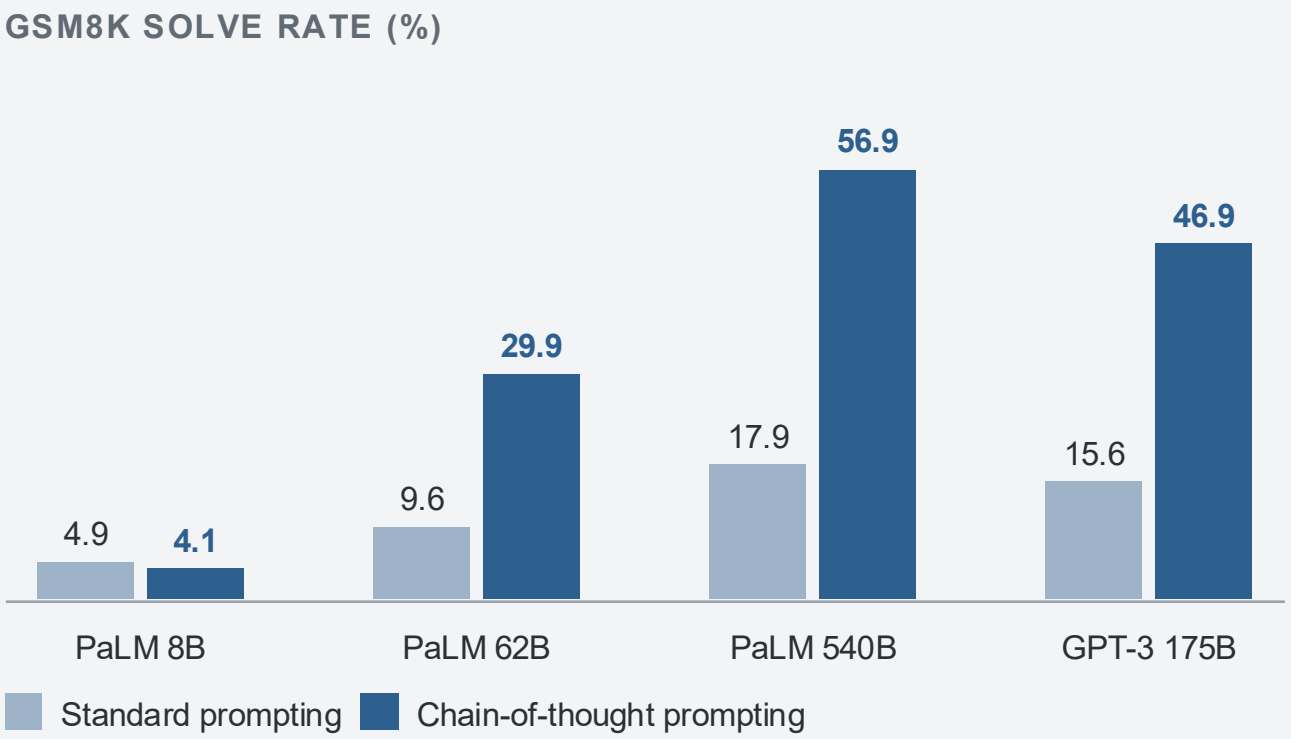
Math, commonsense and symbolic tasks — anything solvable in language.

No training

Elicited from off-the-shelf models by exemplars alone.

WEI ET AL. 2022 · TABLE 2 AND SECTION 3.2

An Emergent Ability of Scale



No gain below ~100B parameters: small models wrote *"fluent but illogical chains of thought."*

Biggest gains on hard multi-step problems; little or none on one-step ones.

PaLM 540B with 8 exemplars (56.9%) beat finetuned GPT-3 175B + verifier (55%).

WEI ET AL. 2022 · SECTIONS 3.3–3.4

Why Does It Work? Three Ablations

Variant	The model writes…	Result on GSM8K
Equation only	just the equation, then the answer	Does not help much (it does help on one- or two-step datasets)
Variable compute only	a row of dots as long as the equation	About the same as standard prompting
Chain of thought after the answer	the answer first, then the reasoning	About the same as standard prompting

Neither extra tokens nor recalling knowledge explains the gain: the **natural-language steps, written before the answer**, do the work.

Robustness. Other annotators, a concise style, or exemplars sampled from GSM8K all beat standard prompting by a large margin.

KOJIMA ET AL. 2022 · SECTION 3 AND TABLE 2

Zero-Shot CoT: “Let’s Think Step by Step”

STAGE 1 · REASONING EXTRACTION

```
Q: [X]. A: Let’s think step by step. → reasoning Z
```

⬇

STAGE 2 · ANSWER EXTRACTION

```
[X′] [Z] Therefore, the answer (arabic numerals) is → answer
```

> Stage 2 is our Chapter 3 extraction problem, solved with a second prompt instead of a regex.

text-davinci-002	MultiArith	GSM8K
Zero-shot	17.7	10.4
Few-shot (8)	33.8	15.6
Zero-shot CoT	78.7	40.7
Few-shot CoT (8)	93.0	48.7

Accuracy (%). Few-shot uses Wei et al.’s 8 exemplars.

KOJIMA ET AL. 2022 · TABLE 4

The Wording of the Trigger Matters

Category	Trigger	MultiArith
Instructive	Let’s think step by step.	78.7
Instructive	First,	77.3
Instructive	Let’s think	57.5
Misleading	Don’t think. Just feel.	18.8
Misleading	Let’s think step by step but reach an incorrect answer.	18.7
Irrelevant	Abrakadabra!	15.5
Irrelevant	It’s a beautiful day.	13.1
None	(zero-shot, no trigger)	17.7

Only triggers that invite reasoning help.

Asking for a wrong answer lands at the no-trigger baseline.

Compare Block III: swapping “Question:” for “Problem:” changed MATH-500 accuracy.

WANG ET AL. · ICLR 2023 · ARXIV 2203.11171

CoT-SC: Chain of Thought + Self-Consistency

Question (abridged from the paper's Table 4): Henry's 60-mile trip has stops at mile 20 and 15 miles before the end. How far apart are the stops?

PLAIN COT · ONE GREEDY PATH

```
… 60 - 20 = 40 miles … The answer is 40.  wrong
```

COT-SC · SAMPLE M PATHS, THEN VOTE

```
Path 1: … 60 - 20 - 15 = 25 miles. The answer is 25.
```

```
Path 2: … 60 - 15 = 45 … 45 - 20 = 25 … The answer is 25.
```

```
Path m: …
```

```
Majority vote over the final answers → 25  correct
```

```
a* = argmax_a sum_i 1[a_i = a]
```

The reasoning paths are marginalized out; only the final answers vote.

- Sample instead of greedy decoding: 40 paths, T = 0.7, top-k 40 (PaLM-540B).
- GSM8K, PaLM-540B: 56.5% → 74.4%.
- Cost: m generations per question; 5–10 paths give most of the gain.
- In ReAct: 21 samples at T = 0.7; weak agreement triggers the switch to ReAct.

LIVE CODING 1 · NOTEBOOK PART 1

Does CoT Help a 0.6B Model?

In 2022, CoT did not help models below ~100B parameters. Qwen3 0.6B is 900× smaller than PaLM 540B — but it was pretrained with chain-of-thought data.

> **Predict before we run:** on the first 10 GSM8K test problems, which wins — standard, few-shot CoT or zero-shot CoT?

LIVE CODING 1 · STEPS 1–2

A Stop String and Three Prompts

```
def generate_until(model, tokenizer, prompt, device,
                   max_new_tokens=256, stop=()):
    input_ids = torch.tensor(tokenizer.encode(prompt),
                             device=device).unsqueeze(0)
    ids, text = [], ""
    for tok in generate_text_basic_stream_cache(
            model=model, token_ids=input_ids,
            max_new_tokens=max_new_tokens,
            eos_token_id=tokenizer.eos_token_id):
        ids.append(tok.squeeze(0).item())
        text = tokenizer.decode(ids)
        hits = [text.find(s) for s in stop if s in text]
        if hits:  # cut at the first stop string
            return text[:min(hits)]
    return text
```

```
def few_shot_cot_prompt(question):
    demos = "".join(f"Q: {q}\nA: {a}\n\n"
            for q, a in WEI_EXEMPLARS)
    return demos + f"Q: {question}\nA:"

def zero_shot_cot_prompt(question):
    return (f"Q: {question}\n"
            "A: Let's think step by step.")
```

standard_prompt uses the same four exemplars with answers only: “The answer is 6.”

LIVE CODING 1 · STEP 3

Run It on 10 GSM8K Problems

```
gsm8k = load_gsm8k_test()    # 1,319 test problems (OpenAI grade-school-math)
results = compare_methods(model, tokenizer, device, gsm8k[:10], max_new_tokens=384)

# inside solve(): zero-shot CoT needs two prompts (Kojima et al.)
p1 = zero_shot_cot_prompt(question)
z = generate_until(model, tokenizer, p1, device, 384, stop=["\nQ:"])
p2 = p1 + z + "\nTherefore, the answer (arabic numerals) is"
y = generate_until(model, tokenizer, p2, device, 16, stop=["\n"])
```

Method	Correct / 10	Tokens / problem
Standard	…	…
Few-shot CoT	…	…
Zero-shot CoT	…	…

Did the 2022 scale finding hold?

What did the accuracy cost in tokens?

YAO ET AL. 2023 · INTRODUCTION

Reasoning Alone, Acting Alone, or Both?

REASON ONLY (COT)

A “static black box”: thoughts come from internal representations, not the world. Result: fact hallucination and error propagation.

ACT ONLY

Can gather information, but without reasoning cannot track progress, recover from surprises or synthesize an answer.

REACT

Reason to act: create, track and adjust plans. **Act to reason:** bring outside information into the thoughts.

> The authors’ analogy is a cook who thinks between actions: *“I don’t have salt, so let me use soy sauce and pepper instead.”*

YAO ET AL. 2023 · FIGURE 1 (HOTPOTQA)

Four Ways to Answer One Question

"Aside from the Apple Remote, what other device can control the program Apple Remote was originally designed to interact with?"

STANDARD · WRONG

```
Answer: iPod
```

REASON ONLY · WRONG

```
… originally designed to interact with Apple TV. Apple TV can be controlled by iPhone, iPad, and iPod Touch. …
```

ACT ONLY · WRONG

```
Search[Apple Remote] → Search[Front Row] → Search[Front Row (software)] → Finish[yes]
```

REACT · CORRECT

Thought 1: I need to search Apple Remote and find the program it was originally designed to interact with.
Act 1: Search[Apple Remote]
Obs 1: … originally designed to control the Front Row media center program …
Thought 2: … I need to search Front Row next …
Act 2: Search[Front Row]
Obs 2: Could not find [Front Row]. Similar: [… 'Front Row (software)' …]
Thought 3: Front Row is not found. I need to search Front Row (software).
Act 3: Search[Front Row (software)]
Obs 3: Front Row is a discontinued media center software …
Thought 4: … controlled by an Apple Remote or the keyboard function keys. So the answer is keyboard function keys.
Act 4: Finish[keyboard function keys]

YAO ET AL. 2023 · SECTIONS 2–3.2

ReAct, Formally

An agent sees observation o_t, takes action $a_t \in A$ by policy $\pi(a_t \mid c_t)$, with context $c_t = (o_1, a_1, \ldots, o_{t-1}, a_{t-1}, o_t)$.

ReAct: augment the action space to $\hat{A} = A \cup L$, where L is the space of language.

A thought $\hat{a}_t \in L$ changes nothing outside and returns no observation. It only updates the context: $c_{t+1} = (c_t, \hat{a}_t)$.

SETUP FOR HOTPOTQA AND FEVER

search[entity]: first 5 sentences of the page, or the top-5 similar titles.

lookup[string]: next sentence containing the string (Ctrl+F).

finish[answer]: end the episode.

Frozen PaLM-540B; 6 (HotpotQA) or 3 (FEVER) hand-written trajectories; a thought before every action.

LIVE CODING 2 · NOTEBOOK PART 2

ReAct from Scratch

1 An offline mini-Wikipedia

2 The paper's prompt

3 The loop

4 Test with Figure 1's trajectory

5 Swap in Qwen3 0.6B

> **Predict:** which of the paper's failure modes will a 0.6B model show first?

LIVE CODING 2 · STEPS 1–2

An Environment and a Prompt

```
def step(self, action):  # "Search[Apple Remote]"
    m = re.match(r"^\s*(search|lookup|finish)\[(.*)\]\s*$",
                 action, flags=re.IGNORECASE)
    if not m:
        return f"Invalid action: {action}", False
    kind, arg = m.group(1).lower(), m.group(2)
    if kind == "search":
        return self.search(arg), False
    if kind == "lookup":
        return self.lookup(arg), False
    self.answer, self.done = arg, True
    return f"Episode finished, answer = {arg}", True
```

TRY IT BY HAND (ACTUAL OUTPUT)

```
env.step("Search[Front Row]")
→ Could not find [Front Row]. Similar: ['Front Row
(software)', 'Front Row Motorsports', …]
```

```
env.step("Lookup[function keys]")
→ (Result 1 / 1) Front Row can be controlled by an
Apple Remote or the keyboard function keys.
```

The prompt: an instruction naming the three actions + two of the paper's six HotpotQA trajectories (verbatim, Appendix C.1) + "Question: … Thought 1:"

LIVE CODING 2 · STEP 3

The Loop: Thought → Action → Observation

```
def react(question, env, llm, max_steps=7):
    env.reset()
    prompt = REACT_INSTRUCTION + REACT_EXAMPLES + f"Question: {question}\n"
    for i in range(1, max_steps + 1):
        out = llm(prompt + f"Thought {i}:", stop=[f"\nObservation {i}:"])
        if f"\nAction {i}:" in out:
            thought, action = out.split(f"\nAction {i}:", 1)
        else:  # the model forgot the action: ask for it explicitly
            thought = out.split("\n")[0]
            action = llm(prompt + f"Thought {i}:{thought}\nAction {i}:", stop=["\n"])
        thought, action = thought.strip(), action.strip().split("\n")[0]
        obs, done = env.step(action)
        prompt += f"Thought {i}: {thought}\nAction {i}: {action}\nObservation {i}: {obs}\n"
        if done:
            return env.answer, prompt
    return None, prompt  # no answer: the paper backs off to CoT-SC
```

LIVE CODING 2 · STEPS 4–5

Test with a Script, Then Swap in Qwen3

STEP 4 · SCRIPTEDLLM REPLAYS FIGURE 1 (ACTUAL OUTPUT)

```
Action 1: Search[Apple Remote]
Observation 1: … originally designed to control the Front Row
media center program …
Action 2: Search[Front Row]
Observation 2: Could not find [Front Row]. Similar: ['Front Row
(software)', …]
Action 3: Search[Front Row (software)]
Observation 3: … controlled by an Apple Remote or the keyboard
function keys. …
Action 4: Finish[keyboard function keys]
ANSWER: keyboard function keys
```

STEP 5 · THE REAL MODEL

```
answer, traj = react(FIG1_QUESTION, MiniWikiEnv(MINI_WIKI),
                     qwen_llm(model, tokenizer, device))
```

BLOCK IV · COMPARISON

CoT and ReAct at a Glance

	Chain of thought	ReAct
Knowledge	Internal only, from the parameters	Internal plus observations from tools
Typical failure	Hallucinated facts (56% of CoT failures)	Reasoning loops (47%), uninformative search (23%)
Cost	One generation per question	One generation per step, plus tool calls; the prompt grows each step
Strong on	Math and logic: GSM8K, MATH-500	Knowledge-heavy QA, fact checking, interactive tasks
In this course	Sessions 6–8: sampling, search, RL	Part III: the agent loop

DISCUSSION · 3 MIN

Give Our MATH-500 Model a Tool?

Suppose ReAct could call a calculator or SymPy. Which actions would you add, and what would change in the Chapter 3 verifier?

If the tool does the arithmetic, is the answer still the model's reasoning — and what could go wrong once actions touch the real world?

Block IV in Four Lines

1. Few-shot CoT: exemplars that show their working; in 2022 it helped only models of roughly 100B parameters or more.
2. Zero-shot CoT: one trigger sentence, plus a second prompt to extract the answer.
3. ReAct: thoughts interleaved with actions; grounded observations cut hallucination but add loops and search errors.
4. The best prompting results combined internal reasoning (CoT-SC) with external acting (ReAct).